

A Single-Receiver, Software-Defined RSSI Front End for Multi-Pilot FPV Race Timing at 1 kHz per Pilot

Louis Hitchcock

FPVGate Project

6 October 2026, Revision 4

Abstract

RSSI lap timers for first-person-view (FPV) drone racing normally dedicate one analog receiver to each pilot and sample each receiver at about 1 kHz. This paper describes a replacement front end based on the internal 5 GHz receiver of a single ESP32-C5. The work progressed in two generations. The first-generation receiver tuned the C5 directly, outside the normal Wi-Fi channel API: exact-frequency PHY tuning recovered about 25 dB on Raceband R3 compared with nearest-Wi-Fi-channel tuning, and direct tuning reached Raceband R8 at 5917 MHz even though the standard Wi-Fi path could not. R8 also exposed a measurement problem: the original RSSI path could hold a stale AGC-derived value when no packet-like activity was detected. Enabling the PHY's continuous signal-RSSI mode produced a live reading immediately after retune. The second-generation C5MK receiver replaces scalar RSSI with narrow-band power calculated from raw I/Q captures and time-multiplexes up to eight pilots. A single-pass band-power estimator, a shortened synthesiser retune that removes a fixed 300 μ s delay, and receiver-driven scheduling reduce the per-pilot visit to about 121 μ s. The system delivers about 1030 timestamped readings per second for each of eight pilots, continuously. Raceband R1 to R8 (5658 to 5917 MHz) is validated; frequency limits beyond that are untested. Lap-time accuracy against an independent timing reference remains to be measured.

Keywords: *FPV drone racing; lap timing; RSSI; ESP32-C5; software-defined radio; I/Q sampling; direct PHY tuning; time-division multiplexing; frequency synthesiser; embedded systems*

1 Introduction

FPV drone races can be timed by listening to each aircraft's analog 5.8 GHz video transmitter (VTX). As a quadcopter approaches a gate antenna, received signal strength rises, reaches a peak close to the crossing and then falls. A timing system detects that excursion and records the peak time. Existing low-cost systems commonly dedicate one receiver module to each pilot. RotorHazard [1], for example, uses one RX5808-class receiver per timing node and samples RSSI at approximately 1 kHz. FPVGate [9] originally followed the same one-receiver model.

The ESP32-C5 [3] combines a 240 MHz RISC-V processor with a dual-band Wi-Fi 6 radio. It was designed for communications, but its PHY offers enough low-level control and diagnostic I/Q capture to put the receive chain to other uses. This project began by asking whether the C5 could replace one analog RSSI module. The second question was whether one C5 could replace several receivers by hopping between pilot frequencies fast enough to give every pilot about 1000 readings a second.

The first-generation FPVGateC5RX receiver established how to tune and measure the C5 reliably across the FPV band. The second-generation FPVGateC5MK receiver applies those findings to a multi-pilot architecture. Both generations are described here because several problems that looked like RF limits turned out to be software or measurement problems.

For eight pilots at 1 kHz, one complete visit to a pilot (retune, capture and measurement) must fit into 125 μ s. The initial C5MK retune alone took 398 μ s, and the first host-driven prototype delivered only about 116

samples/s per pilot in short bursts separated by 112 ms blind intervals. The final system reaches about 1030 samples/s per pilot continuously.

The contributions are:

1. Direct exact-frequency control of the ESP32-C5 PHY for FPV reception, including frequencies not exposed by the normal Wi-Fi channel API, with bench evidence showing why exact centring matters on R3 and enabling R8 at 5917 MHz.
2. Identification of a stale scalar-RSSI failure mode on quiet spectrum and use of the PHY continuous signal-RSSI path to obtain a live measurement from the first sample after retune.
3. A narrow-band RSSI estimator built from short raw I/Q captures, including a single-pass implementation mathematically equivalent to the two-pass reference.
4. A fixed 300 μ s wait found inside the vendor synthesiser retune, and a 68 μ s retune without it that reads the same as the original on the bench.
5. A receiver-driven eight-pilot scan with timestamped binary records and host-side clock alignment, replacing a per-pilot host handshake.
6. Integration with FPVGate race logic for single- and multi-pilot operation, plus bench characterisation of throughput, passband, image rejection and post-retune behaviour.

2 Background and development lineage

2.1 RSSI gate timing

A gate pass produces a broad RSSI peak (Figure 1). The timer begins a pass when signal exceeds an *Enter* threshold and ends it after the signal falls below a lower *Exit* threshold. The gap between the thresholds provides hysteresis. The crossing time is assigned to the highest reading within the pass; a minimum lap interval rejects implausibly rapid repeats. The first crossing after a race start is Gate 1 (the hole shot).

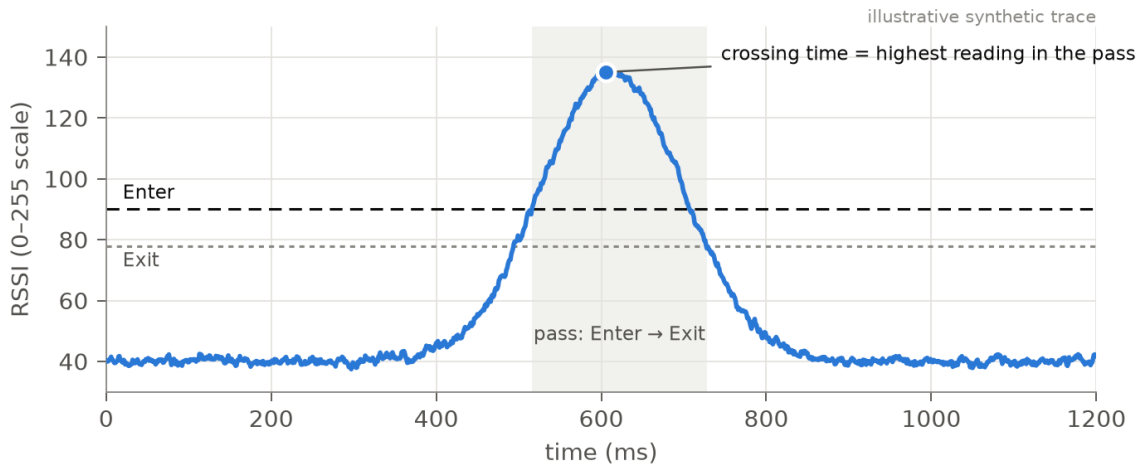


Figure 1. Illustrative RSSI gate pass (synthetic trace). A pass starts when the signal rises above *Enter*, ends when it falls below *Exit*, and is timed at its highest reading.

2.2 The single-receiver timing constraint

With N pilots served in turn, each pilot is visited once per scan cycle. Its sampling rate is approximately

$$f_{\text{pilot}} = 1 / (N \times T_{\text{visit}}) \quad (1)$$

For $N = 8$ and a 1 kHz target, T_{visit} must be no more than 125 μ s. Wi-Fi radios normally change channel occasionally, not thousands of times a second, so software overhead and synthesiser settling both matter.

2.3 Raw I/Q access on the ESP32-C5

The C5 PHY contains a diagnostic sample-capture engine that writes consecutive baseband I/Q samples into on-chip SRAM. The existence of raw I/Q capture on ESP32 radios was publicly demonstrated by ESP-SDR [5] and used for FPV reception by C5VRX [6]. This implementation was written independently, from Espressif's Apache-2.0 PHY and RF-test libraries, the C5 datasheet, a facts-only specification and bench measurements. It contains no source code from those GPL projects.

2.4 Generation 1: discovering the real RF constraints

The first-generation receiver initially used the standard Wi-Fi channel path and treated the nearest legal Wi-Fi channel as the tuning point. Bench testing exposed two different problems that looked like weak or slowly settling RF but had different causes (Figure 2, Table 1).

R3 (5732 MHz) is 12 MHz from Wi-Fi channel 144. With nearest-channel tuning, the project recorded about -74 dBm and a highly unstable mapped output. Direct PHY tuning to 5732 MHz produced about -49 dBm and a stable output on the same bench setup, roughly a 25 dB recovery without changing gain. The fix was exact frequency centring; no extra gain was needed.

R8 (5917 MHz) lies above the highest 5 GHz channel accepted by the ordinary Wi-Fi tuning path used in the first-generation firmware. Disassembly showed that the lower PHY operates on an MHz frequency value and that the fractional-N PLL arithmetic itself did not contain the same channel-table ceiling. Direct PHY tuning placed the radio at 5917 MHz. A sweep with a VTX on R8 peaked around 5912-5920 MHz, consistent with the requested frequency.

R8 then exposed a separate RSSI issue. The original scalar RSSI function combined a noise-floor term with an AGC-derived value that was not continuously refreshed in quiet spectrum. With no packet-like activity the reading could remain at its previous value, creating the appearance of slow RF settling or a stuck-high baseline. Enabling the PHY's continuous signal-RSSI mode and reading the corresponding live value changed R8 from a frozen measurement to approximately -48 dBm with the VTX on and -94 dBm with it off, live from the first sample in that test.

Continuous signal RSSI also made background Wi-Fi bursts visible on in-band channels. The first-generation firmware used short-duration spike suppression before a peak hold to reduce those false rises. In the multi-pilot generation, a frequency-selective I/Q band-power estimator replaces the scalar signal-RSSI measurement, which suits scanning several known frequencies better.

Table 1. First-generation findings that enabled the C5MK multi-pilot design. Values are bench observations from the FPVGateC5RX development log [10].

Case	Original symptom	Root cause	Project fix	Bench result
R3, 5732 MHz	Weak/unstable reading	Carrier 12 MHz off-centre using nearest Wi-Fi channel	Direct PHY tune to exact MHz	≈ -74 dBm to ≈ -49 dBm
R8, 5917 MHz	Normal API could not tune; reading later appeared to settle slowly	Driver channel-table limit plus stale scalar RSSI	Direct PHY tune + continuous signal-RSSI mode	≈ -48 dBm VTX on; ≈ -94 dBm off

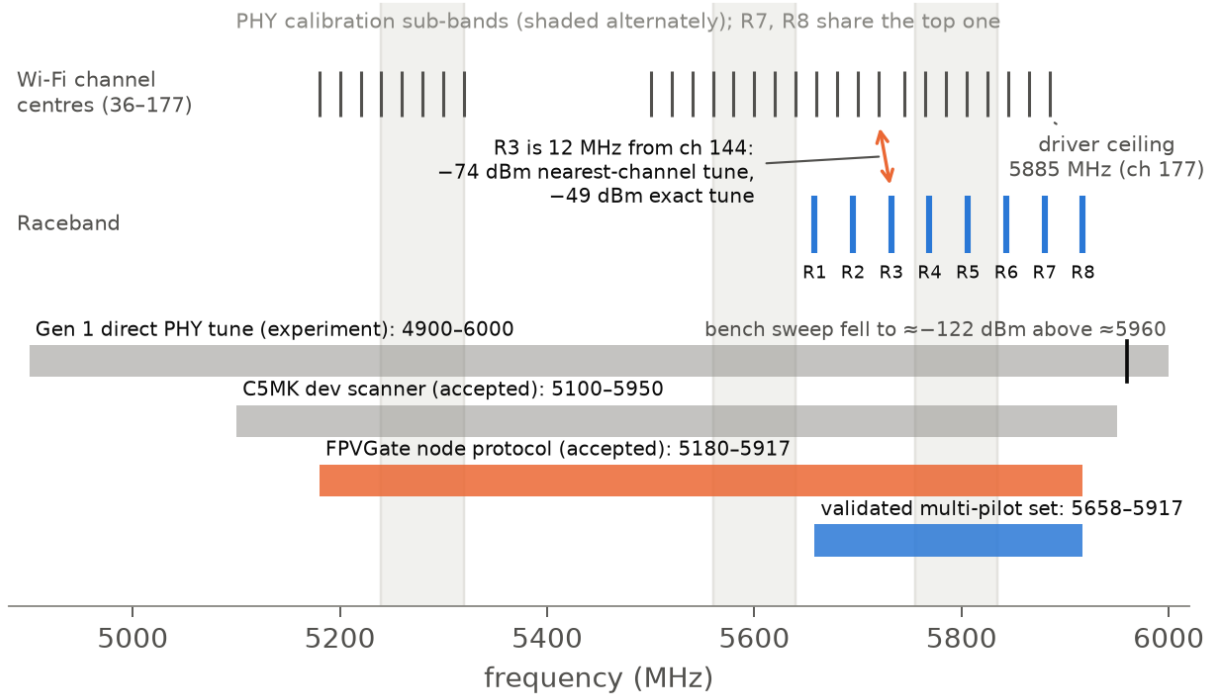


Figure 2. Raceband against the Wi-Fi channel grid, the PHY calibration sub-bands and the software tuning ranges. R3 sits 12 MHz from the nearest Wi-Fi channel centre and R8 lies above the driver's 5885 MHz ceiling. Only R1 to R8 is validated for the multi-pilot system.

3 System architecture

The second-generation system uses two processors and a browser client (Figure 3). The ESP32-C5 is the RF front end: it cycles through the configured pilot frequencies and produces timestamped signal measurements. The ESP32-S3 is the race controller: it receives those records over UART, detects gate passes, runs race logic, stores results and serves the browser application.

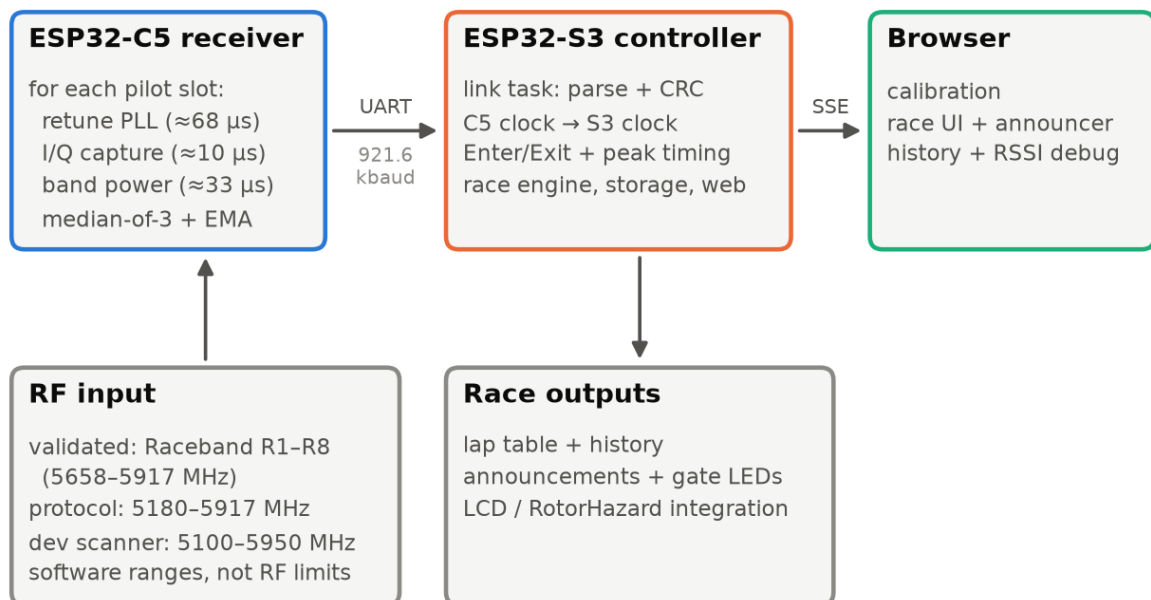


Figure 3. System architecture. Raceband R1 to R8 is validated. The development scanner accepts 5100 to 5950 MHz and the FPVGate node protocol 5180 to 5917 MHz.

Table 2. Division of work between receiver and controller.

	ESP32-C5 receiver	ESP32-S3 controller
Function	Measure each pilot signal	Lap timing, races, UI, storage
Per-pilot work	Retune, I/Q capture, band power, filtering	Clock alignment, pass detection, lap rules
Rate	~1030 readings/s per pilot (8 pilots)	~8200 readings/s processed
Interfaces	UART to S3; USB for development	UART to C5; Wi-Fi web application

4 Methods

4.1 Frequency placement and receiver gain

For each pilot at frequency f_p , the local oscillator is normally placed 10 MHz below the carrier. This moves the signal away from the receiver's DC region while keeping it inside the measured passband. Unlike the first-generation nearest-channel method, the multi-pilot path programs the low-level synthesiser directly and treats the requested MHz value as the tuning target. The receiver gain is forced to index 30; the Wi-Fi packet AGC is not used.

The current C5MK development scanner accepts integer-MHz pilot entries from 5100 to 5950 MHz. The FPVGate node protocol accepts 5180 to 5917 MHz and includes R8. These are software limits. Only the eight Raceband channels have been tested with the multi-pilot measurement chain, so neither range says anything about the radio's RF limits.

4.2 I/Q acquisition

A capture of $n = 256$ consecutive samples ($3.2 \mu\text{s}$ at 80 MS/s) is triggered directly through the sample engine's control registers, bypassing a vendor capture path that adds approximately 1.7 ms of overhead. The capture buffer is prefilled with a sentinel and guarded so incomplete or overlong transfers can be detected cheaply.

4.3 Band-power estimation

Each capture $x_i = I_i + jQ_i$ is reduced to the power in a band of about 10 MHz centred on the pilot. Samples are grouped into blocks of eight, multiplied by a Hann window [7] and mixed at the pilot's baseband offset. The squared magnitude of the block result is averaged across the capture. DC is removed using the capture mean.

$$P = (s / B) \times \sum_b \left| \sum_k w_k (x_{bL+k} - \bar{x}) \cdot \exp(-j2\pi f_{bb} (bL+k) / f_s) \right|^2 \quad (2)$$

A straightforward implementation needs one pass to obtain the mean and another to compute the corrected block sums. Because the mean correction is linear, the uncorrected block can be accumulated first and corrected afterwards. When $f_{bb}L / f_s$ is an integer ($10 \text{ MHz} \times 8 / 80 \text{ MS/s} = 1$ in the operating configuration), every block reuses the same coefficients. The resulting single-pass implementation is bit-for-bit equivalent to the two-pass reference on the project's synthetic test cases and reduces the 256-sample measurement from about $66 \mu\text{s}$ to $33 \mu\text{s}$.

4.4 Fast retuning

The original retune used Espressif's `phy_set_rf_freq_offset` path and took a constant $398 \mu\text{s}$. Disassembly showed that the synthesiser programming and calibration work is followed by a fixed $300 \mu\text{s}$ busy-wait before two additional 5 GHz updates (Figure 4, Table 3). The internal steps are separately exported by the library. C5MK executes the same sequence without the fixed $300 \mu\text{s}$ delay and, in the fastest scan path, omits the two optional post-delay updates whose state is already established by the preceding setup. The complete shortened path is about $103 \mu\text{s}$; the scan path is about $68 \mu\text{s}$.

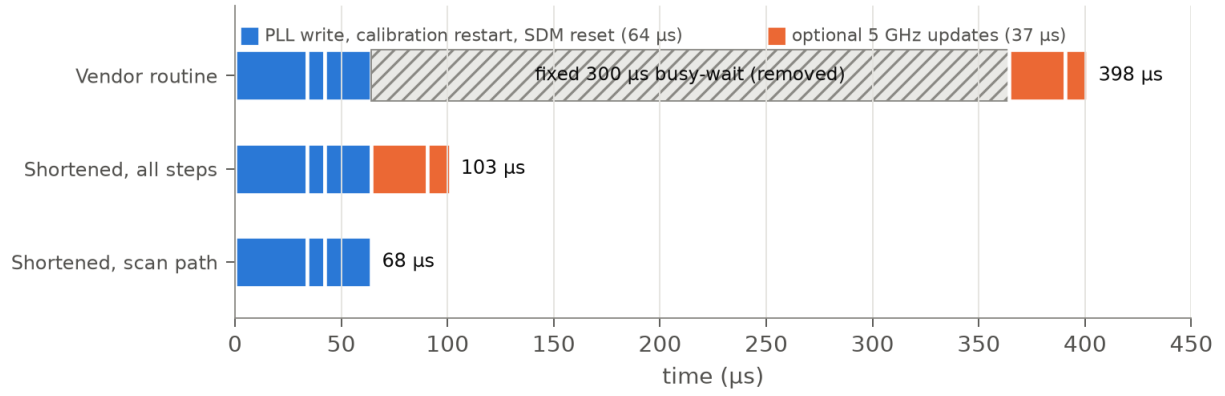


Figure 4. Synthesiser retune drawn to scale. Bars show each step timed individually; labels give the whole call timed as one, which differs by a few microseconds of timing overhead.

Table 3. Retune components measured during C5MK development.

Retune step	Measured time
Synthesiser settings write	33.8 µs
Restart calibration	8.3 µs
Sigma-delta reset	22.1 µs
Fixed vendor wait (removed)	300 µs
5 GHz clock update (optional)	26.4 µs
5 GHz frequency-memory update (optional)	10.7 µs
Total, no fixed wait	103 µs
Total, optional steps omitted	68 µs

4.5 Receiver-driven scheduling and serial link

The first multi-pilot prototype was host-driven: the S3 requested a tune, waited for state changes, collected a 16 ms slot and then moved to the next pilot. The handshake cost about 2.1 ms and each pilot was blind for the other seven slots. The final design sends the pilot list once; the C5 then scans continuously and returns one binary record per cycle (Figure 5). Each record carries a cycle sequence number, a receiver timestamp at the middle of the first capture, a pilot mask and a 10-bit value plus per-pilot time offset (Appendix A). CRC-8 protects the record.

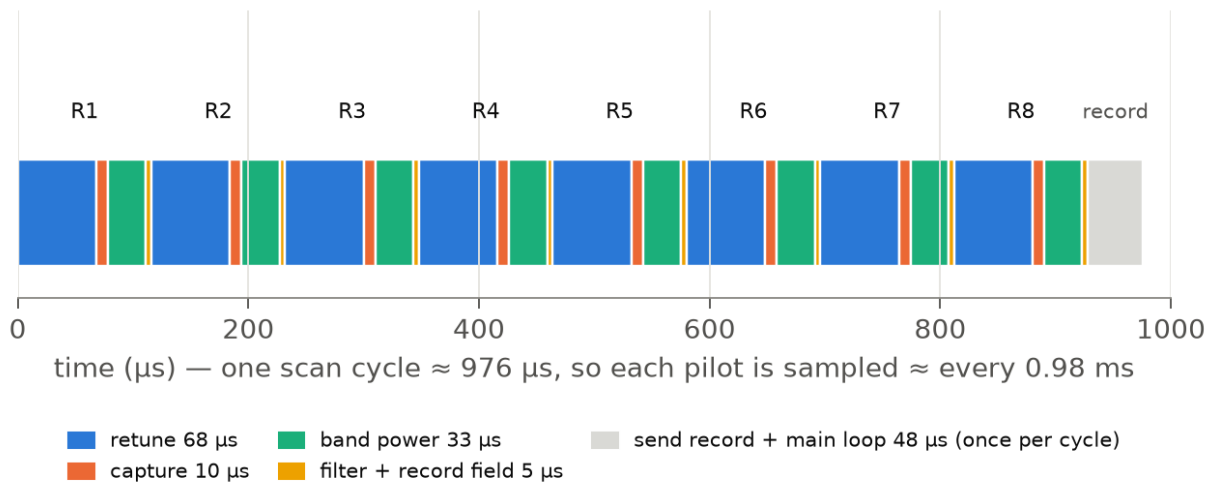


Figure 5. One receiver scan cycle with eight Raceband pilots, from per-visit profiling. The cycle takes about 976 µs, so each pilot is sampled about once every 0.98 ms.

The S3 aligns the C5 clock to its own clock using a minimum-latency offset estimate that is allowed to drift slowly. Because the capture time is generated on the receiver, UART latency and controller scheduling latency do not directly shift a lap timestamp.

4.6 Controller processing and race integration

A high-priority controller task services the link approximately every millisecond, reads the UART in bulk, validates records and queues readings per pilot. A light smoothing stage is followed by Enter/Exit pass detection and peak timing. One-pilot operation is passed through the existing FPVGate lap engine; multi-pilot operation maintains independent pass state and lap history for each configured pilot and exposes it through the same browser interface.

5 Experimental setup

Generation 1 tuning/RSSI tests:

- ESP32-C5 receiver board at approximately 1 m from a bench analog VTX; direct Wi-Fi-channel and direct-PHY tuning compared on the same setup.
- R3 validation compared nearest Wi-Fi-channel tuning with exact 5732 MHz tuning.
- R8 validation used direct tuning to 5917 MHz, live scalar RSSI and a frequency sweep around the carrier.

Generation 2 multi-pilot tests:

- Receiver: ESP32-C5-Zero-class board, chip revision 1.0, ESP-IDF 5.5.1; gain index 30, wide analog bandwidth mode.
- Controller: Sseed Studio XIAO ESP32-S3 running FPVGate, UART at 921600 baud.
- Signal source: bench 5.8 GHz analog VTX, operated in 30-60 s bursts.
- Validated channels: Raceband R1-R8, 5658-5917 MHz.
- Instrumentation: microsecond timer and cycle counter on the C5, hop/scan profiling commands, controller-side sequence/drop/error counters, and 60 s validation runs.

6 Results

6.1 Exact-frequency tuning: R3

R3 showed most clearly how the Wi-Fi channel API could create what looked like a sensitivity problem. The nearest legal Wi-Fi centre was 12 MHz away. The same VTX and receiver produced approximately -74 dBm with that method and approximately -49 dBm with the PHY tuned directly to 5732 MHz. No gain increase was needed, so the loss came from frequency placement. The first-generation auto-selection logic therefore switched channels that were more than 5 MHz from a Wi-Fi centre to the direct-PHY method.

6.2 R8: driver limit, direct tuning and live RSSI

The standard first-generation Wi-Fi path stopped below R8, but direct PHY control tuned the receiver to 5917 MHz and the R8 VTX produced the expected response. A 4 MHz-step validation sweep peaked around 5912-5920 MHz. Above roughly 5960 MHz the readings in that setup fell towards an unusable level. That is a single bench observation and was not investigated further.

The more important finding was that the original scalar RSSI path was stale in quiet spectrum. Enabling continuous signal-RSSI mode eliminated the apparent 0.2-3.5 s settling behaviour: in the logged R8 test the live value was about -48 dBm with the VTX on and -94 dBm with it off from the first reading. It also changed how earlier ‘settling’ tests had to be read: some of them had measured a stale reading, not the radio.

6.3 Baseline C5MK costs

Table 4. Per-visit costs before optimisation.

Capture n	2048	1024	512	256
Retune	398 μ s	398 μ s	398 μ s	398 μ s
Direct capture	55 μ s	29 μ s	16 μ s	10 μ s
Band power (two-pass)	422 μ s	219 μ s	117 μ s	66 μ s
Per-pilot rate, 8 pilots	139 Hz	186 Hz	225 Hz	251 Hz

6.4 Retune settling

To test whether the vendor's 300 μ s wait was necessary, the VTX was placed on Raceband 1. The receiver retuned onto the carrier from 37 MHz below using the shortened sequence, inserted an additional 0-300 μ s pause and then captured 256 samples. Across twelve repetitions per delay, readings with and without the optional 5 GHz steps matched the steady-state reference to within about 0.2 dB from 0 μ s onwards, with comparable spread. The synthesiser was therefore settled by the time the 68 μ s path returned in this test.

6.5 Post-retune transients

Table 5. Isolated high readings after retune. They occur even with the vendor routine and are removed by the median-of-three filter.

Retune method	Carrier on: >6 dB high	Carrier off: >6 dB high
No retune	0 / 500	0 / 500
Vendor routine, 398 μ s	4 / 500	0 / 500
Shortened, all steps, 103 μ s	10 / 500	0 / 500
Shortened, optional steps omitted, 68 μ s	4 / 500	0 / 500

6.6 Passband and image rejection

With a carrier on R1, the LO was stepped from 38 MHz below to 38 MHz above the carrier while measuring the desired and mirror frequencies (Figure 6). The response is approximately flat to ± 15 MHz, about 1.3 dB down at ± 18.5 MHz, about 3 dB down near ± 21 MHz and near the floor beyond ± 35 MHz. Image rejection is about 18 dB. This prevented the planned 'two pilots per capture' optimisation: putting the LO halfway between two Raceband channels places each pilot close to the other's image, with insufficient rejection.

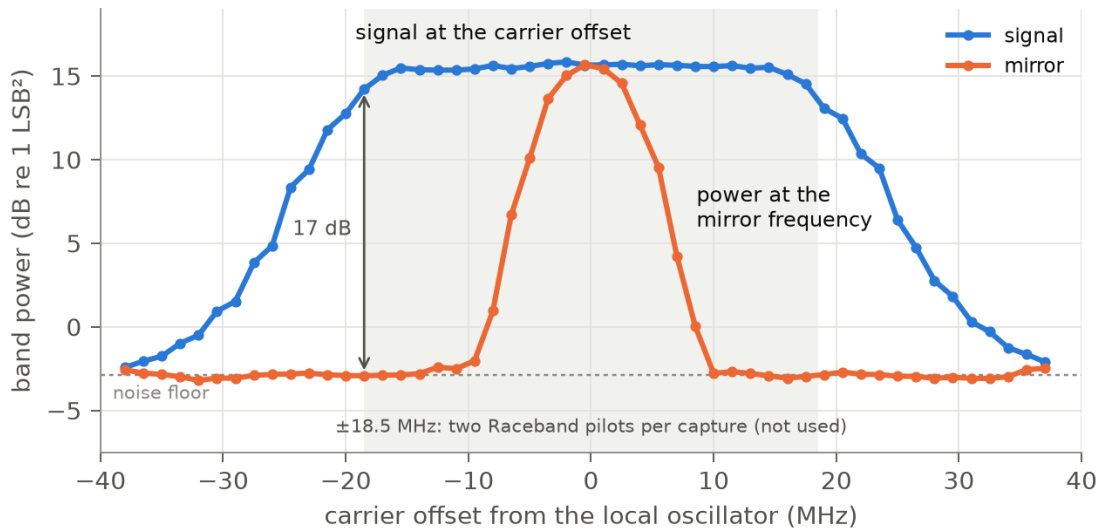


Figure 6. Measured passband and mirror response (carrier on R1, gain 30).

6.7 Throughput

Moving scheduling from the host to the receiver produced the largest throughput gain by removing tune handshakes and the 112 ms blind gaps (Figures 7 and 8). Later improvements removed blocking USB debug output, reduced signal-processing cost and changed S3 UART servicing from byte-at-a-time calls to bulk reads.

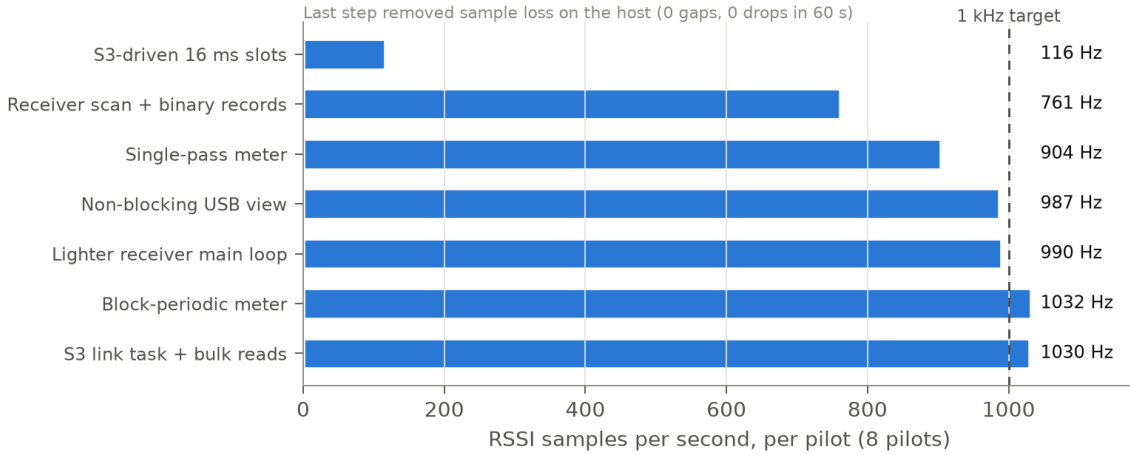


Figure 7. Per-pilot rate through successive C5MK optimisations. The final measured result is approximately 1030 samples/s per pilot.

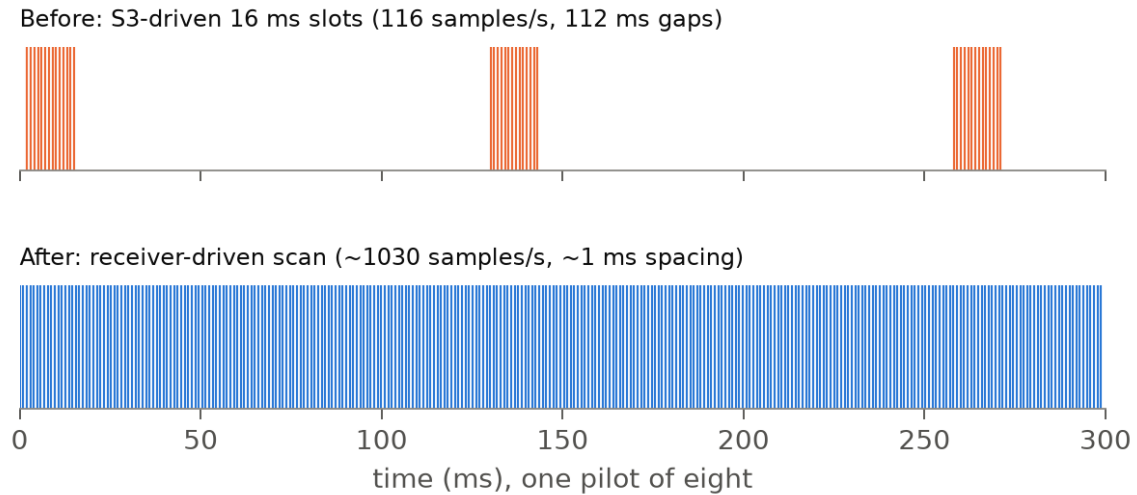


Figure 8. One pilot before and after receiver-driven scanning. The host-driven design observes a 16 ms slot and then misses approximately 112 ms; the receiver-driven design returns about one measurement per millisecond.

Table 6. Final 60 s link measurement with eight Raceband pilots.

Measure	Value
Samples/s per pilot	1031
Samples/s total	8250
Sequence gaps	0
Queue drops	0
Bad records	0
Online / ready	100% / 100%

6.8 Timing budget

The final per-pilot visit is approximately 121 μ s: 68 μ s retune, 10 μ s capture, 33 μ s band power, about 4.8 μ s of between-visit filtering/record work and roughly 6 μ s per visit when once-per-cycle work is amortised (Figure 9). The eight-pilot 1 kHz budget is 125 μ s, leaving only about 3% headroom.

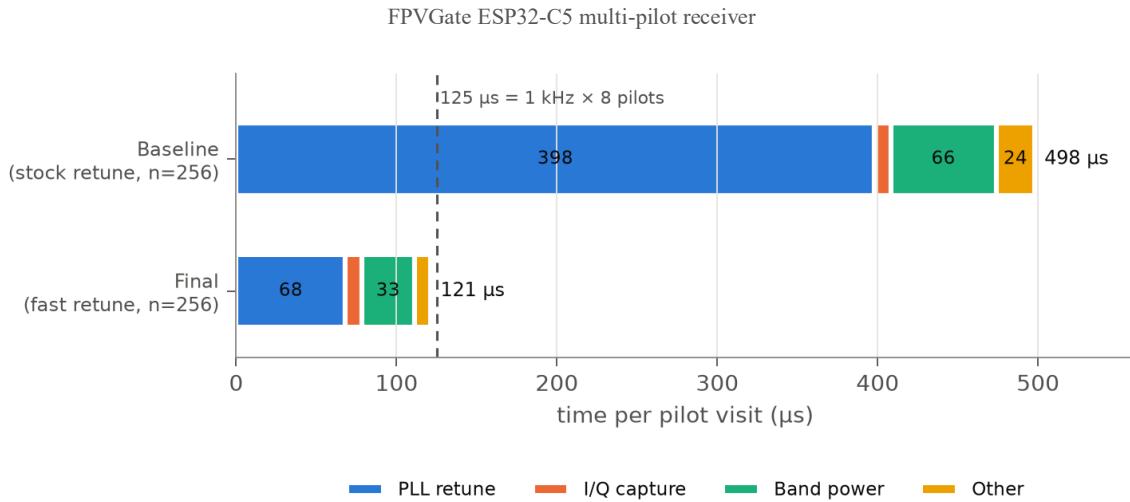


Figure 9. Per-pilot timing budget before and after optimisation.

6.9 Controller servicing

With bulk reads the S3 link task normally polls at roughly 1.3 ms. A periodic controller stall of about 60 ms every 5 s, occasionally reaching 130 ms, has been observed. A 130 ms stall can overflow the 4 KB UART receive buffer and lose a record; the cause has not been found yet.

7 Discussion

The development history shows three different classes of apparent RF problem. R3 was a frequency-centring problem caused by the channel abstraction. R8 initially exposed a driver range restriction, then a stale measurement path that imitated slow settling. C5MK's main timing problem was a fixed software delay inside a vendor retune routine. Each of these looked like an RF limit at first, and each needed a different fix.

Useful reception has been demonstrated on all eight Raceband channels, including R8, and the development scanner accepts 5100 to 5950 MHz. Sensitivity, filter response, calibration and PLL lock margin across that whole range have not been measured. The real limit is where the synthesiser, calibration or analog receive chain stops giving useful, repeatable sensitivity, and finding it needs a dedicated experiment.

Raw I/Q capture on ESP32 radios is not new: ESP-SDR published it first, and C5VRX first showed FPV reception on the C5. What this work adds is exact-frequency tuning and the live-RSSI fix in the first generation, then a fast retune and narrow-band I/Q measurement that keep eight Raceband pilots at about 1 kHz each.

A high sampling rate does not by itself make an accurate lap timer. Gate-peak shape, multipath and antenna geometry will dominate some timing errors even when samples are spaced at 1 ms. Receiver-side timestamps remove serial latency from the measurement, but the system still requires an independent lap-time accuracy study.

7.1 Limitations and threats to validity

- No ground-truth lap-time study yet: results have not been compared with high-speed video or an independent timing system.
- Bench testing dominates the evidence: one or a small number of transmitters/boards, limited temperature variation and no tests with several aircraft flying at once.
- Fast-retune settling was checked narrowly, principally with a carrier approached from a 37 MHz hop. The large R8-to-R1 cycle boundary deserves a dedicated carrier test.
- The 68 µs path relies on state established by prior setup and omits two post-delay updates. It has behaved normally in scanning but remains an undocumented use of internal library functions.
- A small fraction of post-retune readings are anomalously high; median-of-three suppresses them, but the cause is not yet isolated.

- Timing headroom is thin: about 121 μs of a 125 μs per-pilot budget.
- Frequency limits are not fully characterised. Raceband R1-R8 is validated for the current system. The 5100-5950 MHz development acceptance range and the earlier 4900-6000 MHz direct-tune experiment are software ranges, not measured RF limits.
- An experimental 802.11p/ITS-G5 receive mode above approximately 5.75 GHz was added after an external community tip and has not yet had a controlled A/B test, so no improvement is claimed for it.

8 Reproduction notes

To reproduce the system, build and test it one layer at a time before trying the eight-pilot scan. First, verify exact-frequency tuning on a known carrier and confirm that moving the requested MHz value moves the observed peak. Second, validate the measurement path with the carrier on and off so stale RSSI cannot be mistaken for settling. Third, bring up raw I/Q capture and compare the two-pass and single-pass estimators on synthetic and live signals. Fourth, characterise the shortened retune with explicit post-hop delays before enabling the receiver-driven scanner. Finally, validate the timestamped link and race logic.

For RF-range experiments, use a known CW source or synthesiser through coax and attenuation where possible. Sweep frequency and record three boundaries separately: whether the PLL appears to tune, whether sensitivity remains useful and repeatable, and whether RSSI/band-power calibration remains valid. Each can fail at a different frequency.

The C5MK firmware, its documentation and a reference host library are published at github.com/LouisHitchcock/FPVGateC5MK under CC BY-NC-SA 4.0, with commercial licences available separately. Third-party components keep their own licences.

9 Future work

- Ground-truth lap-time study against 240 fps video and an independent RotorHazard timing setup with several aircraft in flight.
- Controlled RF sweep with a calibrated source above and below Raceband to separate PLL tuning range, useful sensitivity range and calibration validity.
- Repeat R3/R8 and retune tests across multiple ESP32-C5 boards and temperatures.
- Root-cause analysis of the post-retune high readings and the periodic S3 controller stall.
- Digital I/Q imbalance correction to improve image rejection; if sufficient, revisit two-pilots-per-capture scanning.
- Evaluate direct register access for further retune reduction while preserving repeatability and calibration.
- A controlled A/B test of the 802.11p/ITS-G5 receive mode.

10 Conclusion

A single ESP32-C5 can act as the RF front end for an eight-pilot FPV lap timer at approximately the same per-pilot sample rate as a one-receiver-per-pilot architecture. The first-generation work showed that the normal Wi-Fi channel abstraction could leave an FPV carrier substantially off-centre and could hide usable frequencies such as R8; it also showed that a stale scalar RSSI path could look like slow RF settling. The second generation built on that with a time-multiplexed design using exact low-level tuning, short I/Q captures, a single-pass band-power estimator, a 68 μs retune and receiver-driven timestamped scanning. The final bench result is about 1030 readings/s for each of eight Raceband pilots. The main open work is to measure lap-time accuracy with several aircraft racing, and to find the radio's real frequency limits.

Acknowledgements and related work

Thanks to C5VRX (Twotoz and contributors), whose discovery that the ESP32-C5 can receive analog FPV video pointed this project in the right direction, and to ESP-SDR (ESPARGOS), who first published raw I/Q capture on ESP32-class radios. RotorHazard provides the open-source reference architecture and the approximately 1 kHz per-pilot benchmark used here. Espressif's PHY and RF-test libraries used for the project's clean-room analysis are distributed under the Apache-2.0 licence. An external community tip from SushiDude (@Ready4Sushi) prompted the project's still-experimental 802.11p/ITS-G5 receive-mode tests; the direct-tuning, R3 and stale-RSSI findings are unrelated to it.

References

- [1] RotorHazard project. *RotorHazard: open-source FPV race timing system*. GitHub repository RotorHazard/RotorHazard.
- [2] RichWave Technology Corp. *RTC6715 5.8 GHz FM receiver datasheet*.
- [3] Espressif Systems. *ESP32-C5 Series Datasheet*.
- [4] Espressif Systems. *ESP-IDF Programming Guide*, release 5.5.
- [5] ESPARGOS project. *ESP-SDR*. Source code and documentation.
- [6] Twotoz and contributors. *C5VRX*. Source code and documentation.
- [7] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51-83, 1978.
- [8] Espressif Systems. *ESP32-S3 Series Datasheet*.
- [9] FPVGate project. *FPVGate: open-source FPV lap timer*. GitHub repository LouisHitchcock/FPVGate.
- [10] FPVGateC5RX project development documentation, `EXTENDED_TUNING.md`, 3 October 2026.
- [11] FPVGate C5MK. Source code and documentation. GitHub repository LouisHitchcock/FPVGateC5MK.

Appendix A. Scan record format

One record is sent per scan cycle from receiver to controller. Multi-byte fields are little-endian. The record begins with 0xA5, which does not occur as the start of the line-oriented control messages, allowing binary scan records and text/status traffic to share the UART.

Table 7. Binary scan record used by the multi-pilot node protocol.

Field	Size	Content
Sync	1 byte	0xA5
Length	1 byte	Payload length
Type	1 byte	'M' scan record
Sequence	1 byte	Cycle counter, wraps at 256
t_0	4 bytes	Receiver time at middle of first pilot capture
Mask	1 byte	Bit i set if pilot slot i included
Per pilot: value	2 bytes	Bits 0-9: 0-1023 reading; bit 15: capture failed
Per pilot: Δt	2 bytes	Microseconds after t_0 for capture midpoint
CRC	1 byte	CRC-8, polynomial 0x07, over length and payload

Appendix B. Glossary

Table 8. Terms used in the paper.

Term	Meaning
FPV	First-person view; the pilot flies using live video from the aircraft.
VTX	Analog video transmitter on the aircraft.
RSSI	Received signal strength indication / signal-strength measurement.
Raceband	Eight common 5.8 GHz FPV channels, R1-R8 = 5658-5917 MHz.
LO	Local oscillator; the receive tuning frequency.
Synthesiser / PLL	Circuit that generates the LO.
I/Q samples	In-phase and quadrature baseband samples from which power at nearby frequencies can be computed.
Passband	Frequency range around the LO that passes with relatively little attenuation.
Image / mirror	Undesired response at the opposite baseband frequency.
Time-multiplexing	Sharing one receiver by visiting several pilot frequencies in sequence.
UART	Two-wire serial link between receiver and controller.
Hysteresis	Separate Enter/Exit thresholds used to avoid repeated pass detection near one level.